UNIVERSIDAD REGIONAL AUTÓNOMA DE LOS ANDES – UNIANDES

Inteligencia Artificial y Big Data

Implementación y evaluación de un sistema de detección de anomalías con aprendizaje automático en flujos de datos financieros del contexto ecuatoriano

Estudiante: Josselyn Pabón

Fecha: Octubre de 2026

Resumen

Este trabajo construye y evalúa un sistema de detección de transacciones fraudulentas pensado para un banco ecuatoriano que procesa alrededor de 3,5 millones de operaciones diarias y que necesita responder en menos de 500 ms por cada lote de 10.000 transacciones. Como no existe un dataset público con datos reales de la banca nacional, se generó un conjunto sintético de 185.484 transacciones en dólares, realizadas por 3.000 clientes de 11 ciudades del país durante septiembre de 2026, con códigos de comercio (MCC), canales digitales (App móvil, Banca web, Transferencias SPI, POS y ATM) y 1.374 casos de fraude (0,74 %) inyectados en forma de ráfagas.

Se implementaron cuatro detectores no supervisados (Isolation Forest, Local Outlier Factor, un autoencoder denso y One-Class SVM) y un clasificador supervisado de gradient boosting como referencia. Sobre los cinco últimos días del mes, que el modelo nunca vio, el autoencoder fue el mejor no supervisado (AUC-PR 0,653 en solo 44 ms por lote), mientras que el modelo supervisado alcanzó un F1 de 0,818 y un AUC-PR de 0,898 con 134 ms por lote. La propuesta final combina ambos: el boosting detecta los patrones de fraude ya conocidos y el autoencoder actúa como red de seguridad ante comportamientos nuevos; esta combinación obtuvo el mejor F1 (0,825) y la menor cantidad de falsas alarmas. El diseño de despliegue se basa en Apache Kafka y Apache Flink, y la explicabilidad con valores de Shapley muestra que la hora del día, el riesgo de la IP, el tiempo desde la última transacción y el uso de un dispositivo nuevo son las variables que más pesan en cada alerta. El informe cierra vinculando estos resultados con las exigencias de la Superintendencia de Bancos, la Ley Orgánica de Protección de Datos Personales y la capacidad real de escalar el sistema en el país.

1. Introducción

En los últimos años el fraude bancario en Ecuador dejó de ser principalmente un problema de tarjetas clonadas en cajeros para convertirse en un problema digital: robo de credenciales de banca móvil, transferencias interbancarias no autorizadas, compras en línea con tarjetas filtradas y pequeños cargos de prueba antes de vaciar una cuenta. El escenario planteado en la tarea refleja esa realidad: el área de Seguridad Informática del banco reporta un aumento del 23 % en los intentos de fraude electrónico en seis meses, y la respuesta esperada es un sistema automático capaz de revisar cada transacción mientras ocurre.

El reto no es solo técnico. Un sistema de este tipo trabaja con una clase extremadamente rara (en nuestros datos, una transacción fraudulenta por cada 134 legítimas), con atacantes que cambian de estrategia, con restricciones de tiempo muy estrictas y con datos personales protegidos por ley. Si el modelo es demasiado sensible, bloquea compras legítimas y el cliente se molesta; si es demasiado permisivo, el dinero se pierde. Por eso el trabajo se organizó en las cuatro fases que pide la guía: preparar y explorar los datos, comparar modelos con métricas adecuadas para datos desbalanceados, diseñar la arquitectura en tiempo real y, finalmente, explicar las decisiones del modelo y conectarlas con la normativa ecuatoriana.

Todo el código está en el cuaderno Jupyter del repositorio. Las cifras que aparecen en este informe son exactamente las que produce ese cuaderno al ejecutarse con la semilla 42, de modo que cualquier lector puede reproducirlas; solo los tiempos en milisegundos cambian según el equipo.

2. Selección y construcción del conjunto de datos

2.1 Por qué un dataset sintético

La guía ofrecía tres opciones. El dataset de Kaggle/ULB tiene 284.807 transacciones europeas de 2013, pero sus variables fueron transformadas con PCA (V1 a V28), así que no hay variables categóricas que codificar, casi no hay valores faltantes y es imposible interpretarlas en términos de negocio, lo que limita mucho el análisis de explicabilidad y la discusión normativa. El IEEE-CIS es más rico, pero sus columnas también están anonimizadas y su tamaño complica trabajar en un equipo personal. Se eligió, por tanto, la **opción (c): un dataset sintético con patrones bancarios ecuatorianos**. Esta opción permite cumplir todos los puntos de la tarea: montos en USD, códigos de comercio locales, ciudades del país, variables categóricas reales, valores faltantes con sentido y una variable temporal que hace posible simular el flujo en streaming.

El dataset se construyó combinando dos herramientas. La librería **Faker**, con configuración en español, generó la parte de identidad: un nombre ficticio para cada uno de los 3.000 clientes y un catálogo de 15 comercios ficticios por cada código MCC, del que se asigna uno a cada transacción. Las transacciones, los montos y los episodios de fraude se generaron con **NumPy y pandas**, porque así se puede controlar con precisión cómo y cuándo aparece el fraude. Faker usa su propia semilla, de modo que no altera los datos generados con NumPy. Los nombres de clientes y comercios dan realismo al dataset, pero no entran a los modelos: un nombre propio no debería influir en una decisión de fraude.

2.2 Cómo se generaron los datos

Primero se crearon 3.000 clientes con ciudad habitual (Guayaquil y Quito concentran el 58 %), edad, antigüedad de la cuenta, tipo de tarjeta, saldo promedio y un segmento (Masivo, Preferente o Pymes) que escala el tamaño de sus compras. Luego, cada cliente realizó un número aleatorio de transacciones en septiembre de 2026, con más actividad entre las 9:00 y las 20:00, en diez tipos de comercio con su código MCC: supermercados (5411), gasolineras (5541), restaurantes (5812), farmacias (5912), servicios básicos (4900), grandes almacenes (5311), transferencias (4829), retiros en cajero (6011), electrónica (5732) y apuestas en línea (7995). Un 6 % de las transacciones normales ocurren en otra ciudad, para representar viajes legítimos y evitar que "estar fuera de casa" sea sinónimo de fraude.

Sobre esa base se inyectaron 520 episodios de fraude de una a seis transacciones seguidas, separadas por pocos minutos. Los fraudes se concentran en la madrugada, en dispositivos nuevos, con intentos fallidos de PIN, IP de alto riesgo y comercios como electrónica, apuestas, transferencias y cajeros. Un 22 % son micro-cargos de 1 a 6 dólares ("prueba de tarjeta") y el 18 % son **fraudes camuflados** que imitan la conducta del propio cliente; estos últimos se incluyeron a propósito para que ningún modelo pudiera llegar a un desempeño perfecto, como sucede en la realidad.

Finalmente se calcularon variables de comportamiento por cliente usando solo información del pasado: transacciones en las últimas 24 horas, minutos desde la transacción anterior, monto promedio histórico, razón entre el monto actual y ese promedio, y distancia en kilómetros entre la ciudad de la transacción y la ciudad habitual.

Tabla 1. *Resumen del dataset generado*

Característica	Valor
Transacciones	185.484
Periodo	1 al 30 de septiembre de 2026
Clientes / ciudades	3.000 / 11 (11 provincias)
Variables del archivo	26 (montos en USD, MCC, canal, ciudad, cliente y comercio con Faker, comportamiento)
Fraudes	1.374 (0,741 %)
Razón de desbalance	1 : 134

Fuente: elaboración propia a partir del cuaderno Jupyter del proyecto.

3. Análisis exploratorio de datos

3.1 Distribución de clases

La primera constatación es el fuerte desbalance: 184.110 transacciones normales frente a 1.374 fraudes. Con esta proporción, un modelo que dijera siempre "normal" acertaría el 99,26 % de las veces y no detectaría ningún fraude. Esta es la razón por la que en todo el trabajo se descarta la exactitud (*accuracy*) como métrica de evaluación.

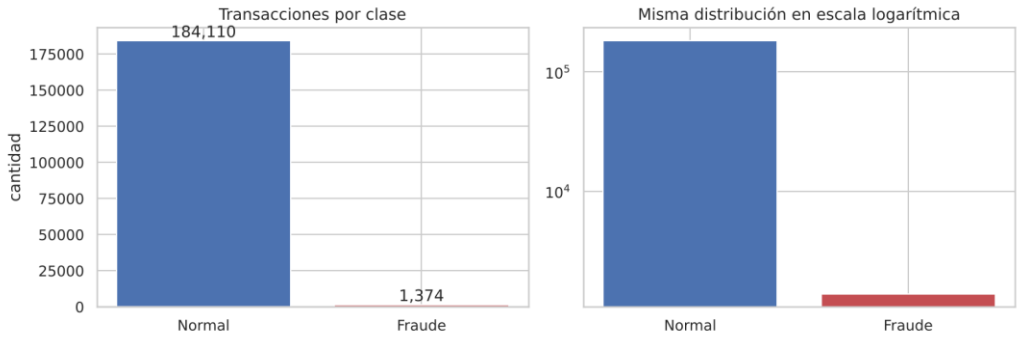


Figura 1. *Distribución de clases en escala normal y logarítmica.*

3.2 Estadísticos descriptivos

El monto tiene una distribución muy sesgada: la mediana es de 27,31 USD, pero la media sube a 61,45 USD y el máximo supera los 11.800 USD. Algo parecido pasa con la razón monto/promedio y con los minutos entre transacciones. Al comparar las clases aparecen diferencias claras, resumidas en la Tabla 2.

Tabla 2. *Media de variables clave por clase*

Variable	Normal	Fraude
Monto (USD)	59,49	324,03

Variable	Normal	Fraude
Razón monto / promedio del cliente	1,15	5,38
Dispositivo nuevo (proporción)	0,02	0,56
Intentos fallidos de PIN	0,04	0,78
Score de riesgo de la IP	0,18	0,52
Distancia a la ciudad habitual (km)	11,0	92,2
Transacciones en las últimas 24 h	2,53	3,14

Fuente: elaboración propia a partir del cuaderno Jupyter del proyecto.

Llama la atención que la variable de ráfagas (transacciones en 24 h) cambia poco en promedio. Esto se explica porque muchos clientes legítimos hacen varias compras al día; la señal útil está más bien en el **tiempo entre transacciones**, que en los fraudes es de pocos minutos.

3.3 Valores faltantes y outliers estructurales

El 64,4 % de los registros no tiene score_ip_riesgo, pero no se trata de un error: en POS y cajeros no existe una dirección IP, así que el faltante es estructural (100 % en esos canales y solo alrededor del 2 % en los canales digitales). El saldo promedio falta en el 3,5 % de los casos y las variables históricas faltan en la primera transacción de cada cliente (1,6 %). Esta distinción es importante, porque el hecho de que falte la IP es en sí mismo una información útil para el modelo.

Con la regla del rango intercuartílico, el 10 % de los montos son outliers, pero solo el 3,7 % de ellos son fraude: la mayoría son compras grandes de clientes Pymes o Preferentes. En cambio, entre los valores atípicos del score de IP el 36 % son fraude y entre los de dispositivo nuevo, el 17,7 %. Es decir, **no todo outlier estadístico es una anomalía relevante**, tal como se explica en el material de clase: un monto alto puede ser perfectamente legítimo.

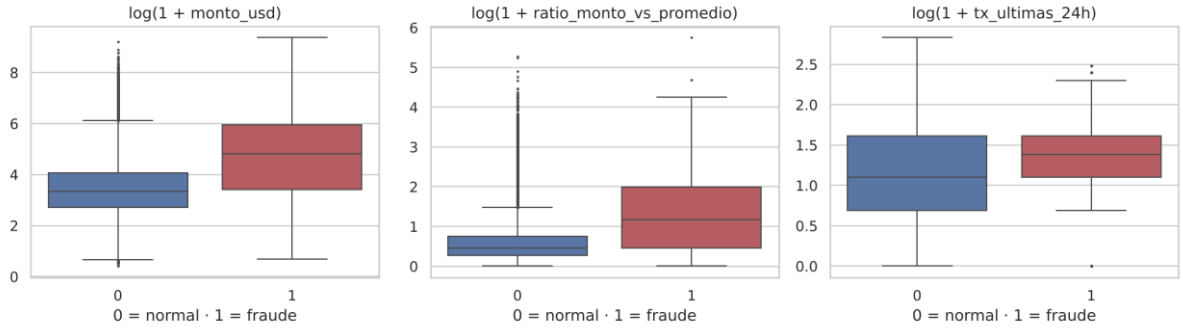


Figura 2. *Monto, razón contra el promedio y ráfagas de 24 h por clase (escala logarítmica).*

3.4 Dónde se concentra el fraude

La tasa de fraude se dispara entre la 1:00 y las 5:00 de la mañana, es mayor en Banca web y en Transferencias SPI que en POS, y los comercios de electrónica y apuestas en línea tienen tasas varias veces superiores a las de supermercados o farmacias. Estos patrones coinciden con lo que se observa en la práctica: los delincuentes prefieren canales sin presencia física y bienes que se pueden revender o retirar con rapidez.

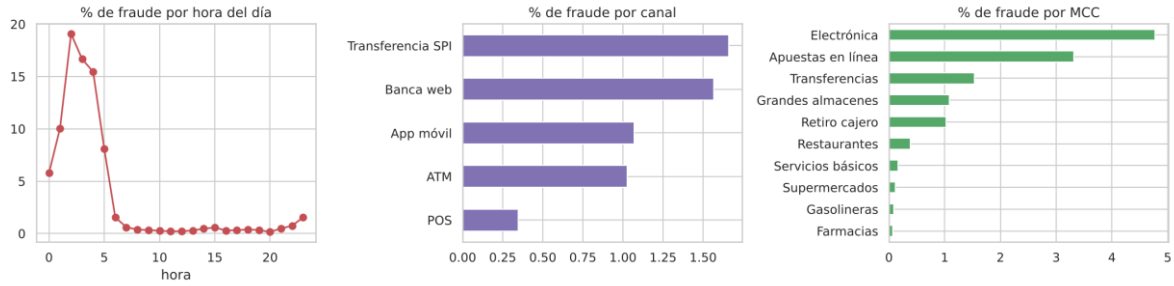


Figura 3. *Porcentaje de fraude por hora, canal y tipo de comercio.*

3.5 Correlaciones

La matriz de correlación de Spearman (elegida por ser robusta ante distribuciones sesgadas) confirma que ninguna variable por sí sola explica el fraude: la más correlacionada es el dispositivo nuevo (0,31), seguida de los intentos fallidos de PIN (0,21), el score de IP (0,17) y la distancia (0,14). El monto, por sí solo, apenas llega a 0,06. Entre las variables explicativas no hay correlaciones altas que obliguen a eliminar alguna, salvo la relación esperable entre el monto y la razón monto/promedio.

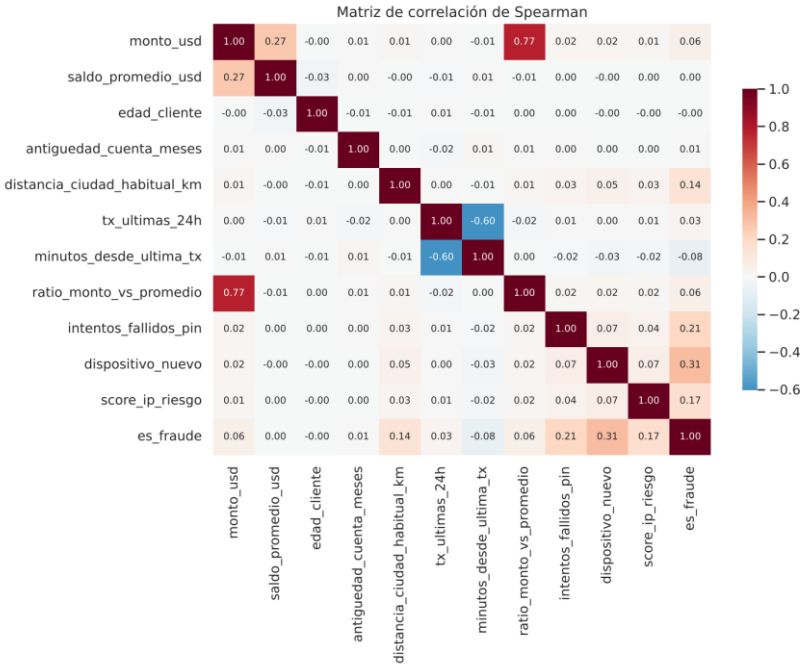


Figura 4. *Matriz de correlación de Spearman.*

4. Preprocesamiento y manejo del desbalance

El pipeline se construyó con un ColumnTransformer de scikit-learn para que exactamente las mismas transformaciones se apliquen en el entrenamiento y en producción. Cada decisión responde a algo observado en la exploración:

- **Partición temporal.** Se entrenó con los días 1 a 21 (130.002 transacciones), se calibraron los umbrales con los días 22 a 25 (24.695) y se evaluó con los días 26 a 30 (30.787, con 250 fraudes). Dividir al azar habría mezclado transacciones de un mismo episodio de fraude entre entrenamiento y prueba, inflando los resultados.

- **Ingeniería de variables.** Se aplicó log(1+x) a montos, razones, distancias y tiempos por sus colas largas; la hora se codificó con seno y coseno para respetar su naturaleza cíclica; y se crearon indicadores de transacción nocturna, fuera de la ciudad y de IP ausente.
- **Valores faltantes.** Imputación con la mediana, que no se ve afectada por los outliers, conservando la bandera de faltante como variable propia.
- **Escalado.** StandardScaler, indispensable para LOF, One-Class SVM y el autoencoder, que trabajan con distancias o gradientes.
- **Variables categóricas.** One-hot para canal, MCC, segmento y tipo de tarjeta, ignorando categorías desconocidas en producción. Se excluyeron deliberadamente los identificadores y la ciudad como texto, para que el modelo no memorice clientes ni penalice a personas por el lugar donde viven.

Para el desbalance se compararon cuatro estrategias con el mismo clasificador y midiendo sobre validación (Tabla 3). Las diferencias son pequeñas (el AUC-PR va de 0,84 a 0,87), lo que indica que el boosting tolera bien el desbalance cuando las variables son informativas. Se eligió **SMOTE-NC** porque está en el grupo con mejor AUC-PR, conserva todas las transacciones reales (el submuestreo descarta el 90 % de las normales) y genera registros sintéticos con categorías válidas. Esto último es importante: el SMOTE clásico interpola todas las columnas, y sobre variables codificadas con one-hot produciría valores sin sentido, como 0,4 de "App móvil". Por eso las variables numéricas se interpolan entre un fraude y su vecino, mientras que las binarias y categóricas se copian del fraude original, que es la idea de SMOTE-NC (Chawla et al., 2002). Para los modelos no supervisados el desbalance se abordó de otra manera: se entrenaron únicamente con transacciones normales, que es el enfoque semisupervisado descrito en la clase.

Tabla 3. *Comparación de estrategias de desbalance (conjunto de validación)*

Estrategia	Filas	AUC-PR	AUC-ROC	F1
Sin tratamiento	130.002	0,873	0,986	0,840
Ponderación de clases	130.002	0,837	0,984	0,832
SMOTE-NC (fraude al 10 %)	141.991	0,864	0,985	0,836
Undersampling (10 % de normales)	13.827	0,862	0,985	0,847

Fuente: elaboración propia a partir del cuaderno Jupyter del proyecto.

5. Modelos implementados

Se implementaron cuatro de los cinco modelos sugeridos más un modelo supervisado de referencia. Todos los no supervisados se entrenaron solo con transacciones normales y su umbral de decisión se fijó en el conjunto de validación, buscando el punto que maximiza el F1. La Tabla 4 resume la configuración.

Tabla 4. *Arquitectura e hiperparámetros de cada modelo*

Modelo	Configuración principal	Datos de entrenamiento
Isolation Forest	200 árboles, 256 muestras por árbol	129.083 normales
Local Outlier Factor	35 vecinos, modo novelty	40.000 normales (muestra)

Modelo	Configuración principal	Datos de entrenamiento
Autoencoder denso	36→24→8→24→36, ReLU, Adam, lote 512, parada temprana (47 épocas)	129.083 normales
One-Class SVM	Kernel RBF, nu = 0,01, gamma = scale	15.000 normales (muestra)
HistGradientBoosting	300 árboles, tasa 0,05, 31 hojas	141.991 filas (con SMOTE-NC)

Fuente: elaboración propia a partir del cuaderno Jupyter del proyecto.

Isolation Forest parte de la idea de que las anomalías son pocas y distintas, así que se aíslan con menos cortes aleatorios. **LOF** compara la densidad alrededor de cada transacción con la de sus vecinos. El **autoencoder** aprende a comprimir y reconstruir transacciones normales; cuando recibe un fraude lo reconstruye mal y ese error es su puntuación de anomalía. **One-Class SVM** traza una frontera alrededor de la región normal. Por último, el **HistGradientBoosting** pertenece a la misma familia que XGBoost y LightGBM y sí utiliza las etiquetas.

El LSTM-Autoencoder no se incluyó. En este dataset cada cliente tiene en promedio unas 60 transacciones al mes, irregularmente espaciadas, y la información secuencial más útil (ritmo de transacciones, promedio histórico) ya se capturó en las variables de comportamiento. Queda como una línea de trabajo futuro cuando se disponga de historiales más largos y de un entorno con aceleración por GPU.

6. Resultados y comparación de modelos

6.1 Por qué estas métricas

Con 250 fraudes entre 30.787 transacciones de prueba, la elección de la métrica cambia por completo la lectura de los resultados. La **precisión** indica qué parte de las alertas son fraudes reales, lo que se traduce en carga de trabajo para los analistas y en molestias para los clientes. El **recall** indica qué parte del fraude se detecta, es decir, cuánto dinero se protege. El **F1** las combina; el F1 macro promedia el F1 de ambas clases y el ponderado lo hace según el tamaño de cada clase, por lo que este último es casi siempre cercano a 1 y aporta poco. El **AUC-ROC** mide la capacidad de ordenar los casos, pero es optimista cuando hay muchísimos negativos (Davis y Goadrich, 2006); por eso se le da más peso al **AUC-PR**, cuya línea base por azar es la tasa de fraude (0,008) y no 0,5 (Saito y Rehmsmeier, 2015). Por último, el **tiempo de inferencia por lote de 10.000** permite verificar el requisito de 500 ms.

6.2 Tabla comparativa

Tabla 5. *Resultados sobre el conjunto de prueba (días 26-30 de septiembre)*

Modelo	Prec.	Recall	F1	F1 macro	F1 pond.	AUC-ROC	AUC-PR	ms / 10k*
Isolation Forest	0,598	0,588	0,593	0,795	0,993	0,961	0,614	151
LOF	0,711	0,540	0,614	0,805	0,994	0,956	0,634	1.357
Autoencoder denso	0,743	0,544	0,628	0,813	0,994	0,967	0,653	44
One-Class SVM	0,694	0,572	0,627	0,812	0,994	0,937	0,637	206
HistGradientBoosting	0,856	0,784	0,818	0,908	0,997	0,994	0,898	134

Modelo	Prec.	Recall	F1	F1 macro	F1 pond.	AUC-ROC	AUC-PR	ms / 10k*
Híbrido HGB + Autoencoder	0,897	0,764	0,825	0,912	0,997	0,978	0,834	142

** Tiempo del pipeline completo (preprocesamiento + modelo) para 10.000 transacciones, mediana de 5 repeticiones en una CPU de 2 núcleos. Fuente: elaboración propia.*

Entre los no supervisados las diferencias son moderadas: todos superan un AUC-ROC de 0,93, pero en AUC-PR se mueven entre 0,61 y 0,65. El autoencoder fue el mejor en AUC-PR y en precisión (de cada 100 alertas, 74 son fraude real) y, además, el más rápido, con apenas 44 ms por lote. El Isolation Forest es el más sensible (recall de 0,588), pero a costa de casi cien falsas alarmas en cinco días. **LOF fue el único modelo que no cumplió la restricción de latencia**: tardó 1,4 segundos por lote, porque debe buscar vecinos en una base de 40.000 transacciones de referencia. One-Class SVM queda en un punto intermedio y su costo de entrenamiento crece casi de forma cuadrática, lo que obligó a usar solo 15.000 filas.

El salto de calidad viene con las etiquetas. El HistGradientBoosting detectó 196 de los 250 fraudes de prueba con solo 33 falsas alarmas, y su AUC-PR de 0,898 está muy por encima del resto. La lectura práctica es clara: si el banco cuenta con historial de fraudes confirmados, un modelo supervisado es la mejor base, y los métodos no supervisados aportan sobre todo como complemento para lo que el modelo supervisado todavía no conoce.

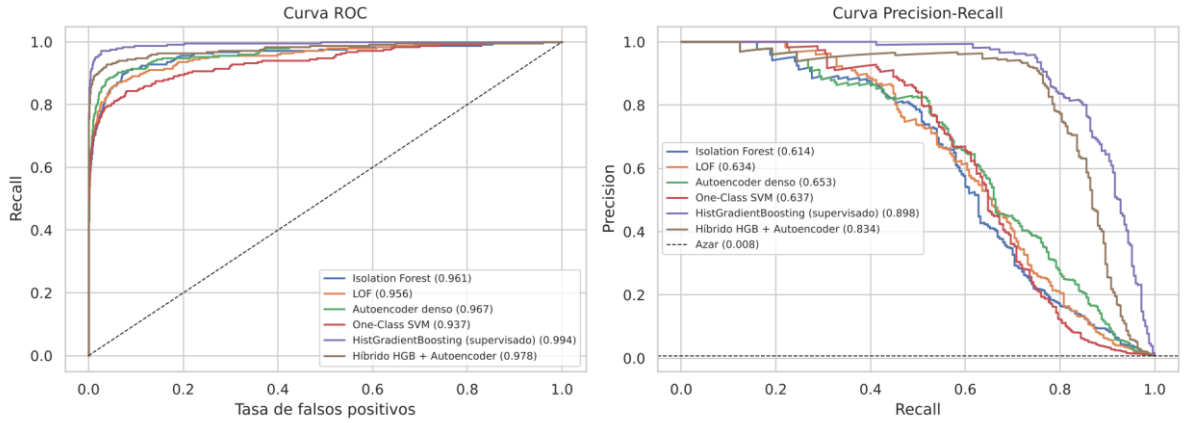


Figura 5. *Curvas ROC y Precision-Recall en el conjunto de prueba.*

La Figura 5 muestra por qué conviene mirar la curva PR: en la curva ROC todos los modelos se agrupan cerca de la esquina superior y parecen casi equivalentes, mientras que en la curva PR se separan con claridad.

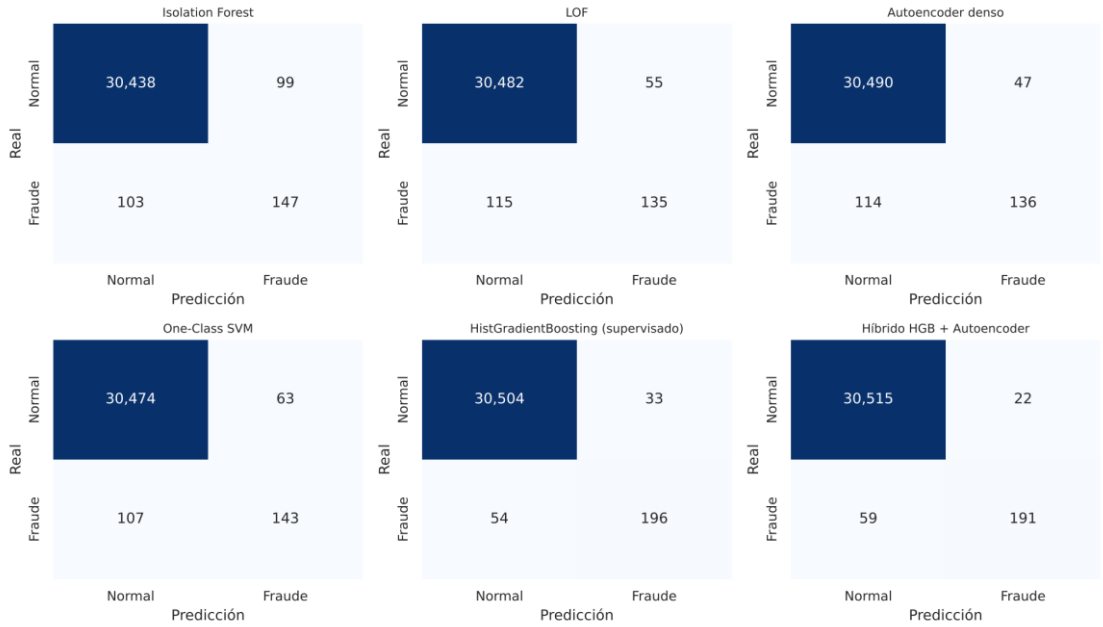


Figura 6. *Matrices de confusión con el umbral calibrado en validación.*

6.3 Modelo seleccionado: relación precisión / latencia

La Figura 7 ubica cada modelo según su AUC-PR y su latencia. Todos, excepto LOF, quedan holgadamente por debajo de los 500 ms. Con ese margen, la latencia deja de ser el criterio que decide y pasa a pesar más la calidad de la detección. Por eso se propone como modelo de producción el **híbrido HistGradientBoosting + Autoencoder**: alerta cuando cualquiera de los dos supera su umbral, y el umbral del autoencoder se fijó muy alto (percentil 99,9 de las transacciones normales) para que solo intervenga ante casos muy extraños.

En la prueba, el híbrido obtuvo el mejor F1 de todos los modelos (0,825 frente a 0,818 del boosting solo): detectó 191 de los 250 fraudes con solo 22 falsas alarmas, frente a 33 del boosting, a cambio de un recall algo menor, y cuesta apenas 8 ms adicionales por lote. Su principal valor, sin embargo, no se ve del todo en esta prueba, porque aquí los fraudes siguen los mismos patrones del entrenamiento; la ventaja aparece cuando surge una modalidad nueva que el boosting nunca vio y que, por lo tanto, no puede reconocer. Este es exactamente el escenario de *concept drift* que describe el material de clase.

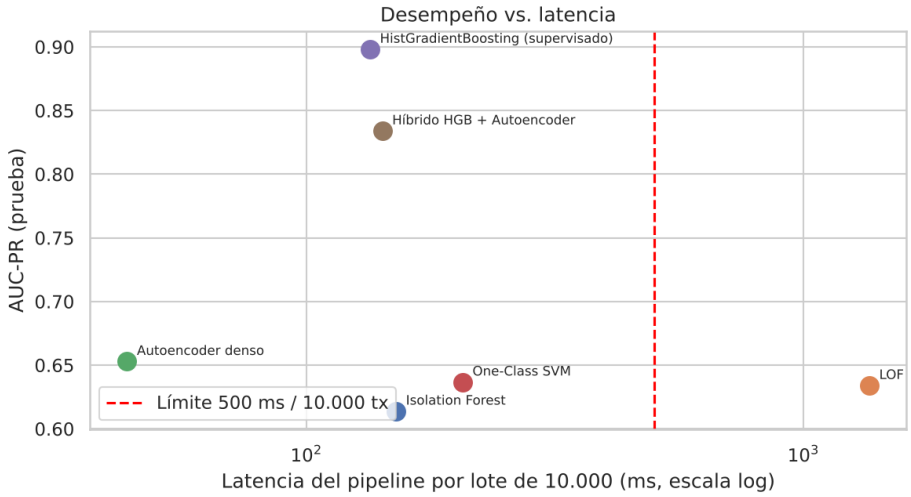


Figura 7. *AUC-PR frente a latencia por lote de 10.000 transacciones.*

7. Diseño del pipeline de detección en streaming

La arquitectura propuesta (Figura 8) sigue el patrón Kappa: un solo flujo de procesamiento que también permite reprocesar el historial reproduciendo los eventos guardados en Kafka. Se prefirió Kappa sobre Lambda porque el caso de uso es esencialmente de alertas en tiempo real y mantener dos bases de código paralelas encarece la operación.

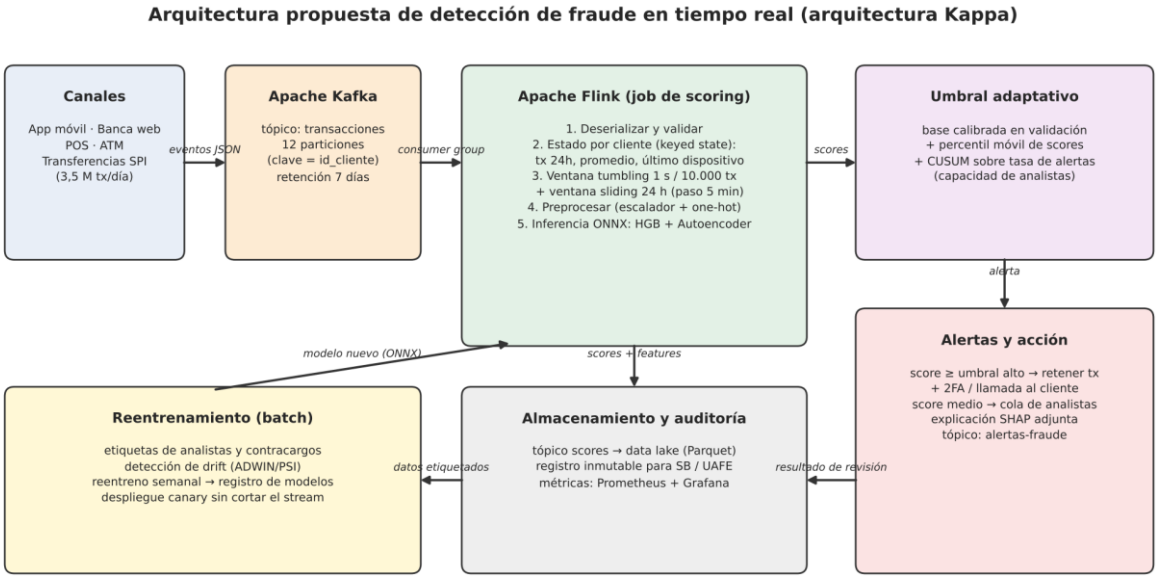


Figura 8. *Arquitectura propuesta con Apache Kafka y Apache Flink.*

1. **Ingesta.** Cada canal (App, Banca web, POS, ATM, SPI) publica la transacción como evento JSON en el tópico transacciones de Kafka, con 12 particiones y el identificador del cliente como clave. Así, todas las operaciones de un mismo cliente llegan en orden a la misma partición, algo indispensable para calcular variables de comportamiento. La retención de siete días permite volver a procesar eventos en caso de falla y sirve como respaldo para auditoría.
2. **Procesamiento con Flink.** Se eligió Flink en lugar de Spark Streaming porque procesa evento a evento, con latencias de milisegundos y un manejo preciso del tiempo del evento; el modelo de

micro-lotes de Spark añade retraso. Flink guarda un estado por cliente (*keyed state*) con su número de transacciones recientes, su promedio y su último dispositivo.

3. **Ventanas.** Una ventana deslizante (*sliding*) de 24 horas que avanza cada 5 minutos mantiene las variables de comportamiento, y una ventana fija (*tumbling*) agrupa las transacciones en micro-lotes que se cierran cada segundo o al llegar a 10.000 eventos, lo que ocurra primero.
4. **Inferencia.** El preprocesador, el boosting y el autoencoder se exportan a formato ONNX y se cargan directamente dentro del operador de Flink, evitando una llamada de red a un microservicio externo.
5. **Umbrales adaptativos.** El umbral base es el calibrado en validación. Sobre él se aplican dos ajustes: el percentil 99 móvil de los scores recientes, que endurece el umbral solo si de repente aparecen muchos scores altos (por ejemplo, una falla en los datos), y un control de capacidad que, junto con un monitoreo CUSUM de la tasa de alertas, impide que el equipo de analistas se sature.
6. **Alertas y acción.** Un score muy alto retiene la transacción y pide una verificación adicional (segundo factor o llamada); un score intermedio va a la cola de analistas acompañado de su explicación SHAP. Todos los scores se guardan en un data lake para auditoría y alimentan el reentrenamiento semanal, que incluye detección de drift (ADWIN o PSI) y un despliegue gradual del nuevo modelo sin detener el flujo.

Para probar la lógica sin un clúster real, en el cuaderno se reprodujeron los cinco días de prueba en ventanas tumbling de 15 minutos (480 ventanas, unas 64 transacciones cada una). La simulación partió exactamente del mismo umbral que se calibró en validación para el híbrido de la Tabla 5. Cada ventana se procesó en 14,1 ms en promedio y el percentil 99 fue de 23,1 ms. El sistema emitió 197 alertas, con una precisión de 0,888 y un recall de 0,700. El recall es algo menor que el de la Tabla 5 porque el umbral móvil sube en las horas en que se concentran los fraudes, que es el precio de proteger al equipo frente a picos de alertas.

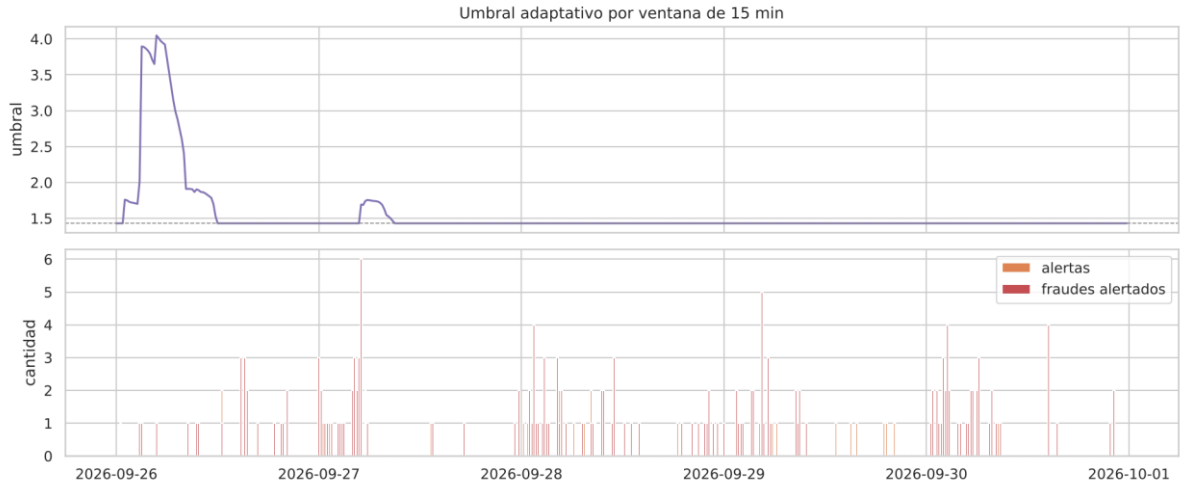


Figura 9. *Evolución del umbral adaptativo y de las alertas en la simulación de streaming.*

8. Latencia y tamaño de lote

Para responder cómo afecta el tamaño del lote, se midió en el cuaderno el tiempo real de preprocesar y puntuar lotes de distintos tamaños con el modelo híbrido, y se sumó un costo fijo estimado de 4 ms por lote para leer y confirmar mensajes en Kafka. Ese costo fijo no se midió: es un valor de referencia razonable para un clúster local.

Tabla 6. *Latencia y throughput según el tamaño del lote (modelo híbrido)*

Tamaño de lote	Latencia por lote (ms)	Throughput (tx/s)
1	14,1	71
100	15,6	6.415
1.000	26,6	37.659
5.000	74,4	67.238
10.000	157,3	63.576
25.000	418,2	59.785

Fuente: elaboración propia a partir del cuaderno Jupyter del proyecto.

El comportamiento es el típico de los sistemas en lote. Con lotes muy pequeños domina el costo fijo de cada llamada: procesar una sola transacción toma casi lo mismo que procesar cien, por lo que el throughput es bajo. A medida que crece el lote, ese costo se reparte y el throughput aumenta, hasta llegar a unas 67.000 transacciones por segundo con lotes de 5.000. A partir de ahí el throughput ya no mejora (las pequeñas variaciones son ruido de medición) y lo único que crece es la latencia, en proporción al tamaño del lote. Con 10.000 transacciones el sistema responde en unos 157 ms, muy por debajo del límite de 500 ms.

Para poner esto en perspectiva: 3,5 millones de transacciones diarias equivalen a unas 41 por segundo en promedio y, suponiendo picos cinco veces mayores en quincenas o feriados, a unas 200 por segundo. En esas condiciones un lote de 10.000 tardaría casi un minuto en llenarse, así que en la práctica conviene cerrar las ventanas por tiempo (cada segundo) y no por cantidad. Así, cada transacción espera como máximo un segundo más unos 20 ms de cómputo, lo que permite decidir antes de autorizar el pago.

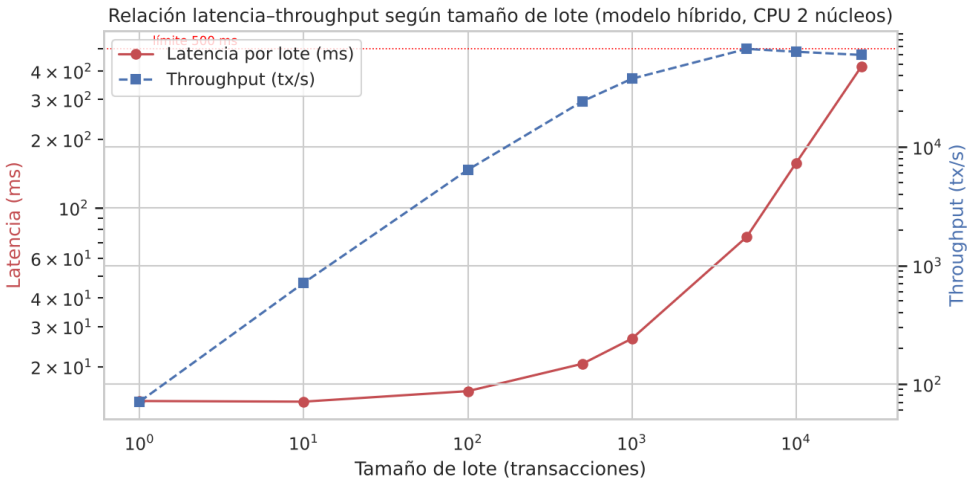


Figura 10. *Relación latencia-throughput según el tamaño de lote.*

9. Interpretación de los resultados y explicabilidad

9.1 Variables más influyentes

Para entender por qué el modelo marca una transacción se calcularon valores de Shapley sobre 300 transacciones de prueba (150 fraudes y 150 normales). Se usó el método de muestreo por permutaciones de Štrumbelj y Kononenko (2014), implementado directamente en el cuaderno. Este método tiene el mismo fundamento teórico que la librería SHAP de Lundberg y Lee (2017), pero no se utilizó esa librería, con el fin de que el cálculo fuera transparente y no dependiera de paquetes externos. Las columnas one-hot se agruparon para explicar cada variable original. La suma de las contribuciones reproduce la predicción con un error medio de 0,18 en log-odds, atribuible al muestreo.

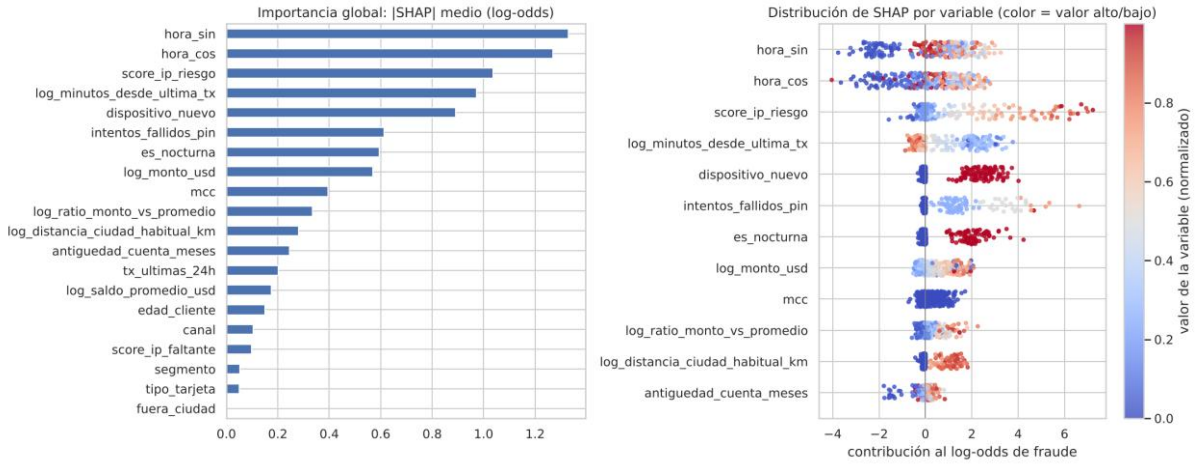


Figura 11. *Importancia global y dirección del efecto según valores SHAP (modelo supervisado).*

Las variables con mayor peso son la **hora del día** (codificada con seno y coseno, 1,33 y 1,27), el **score de riesgo de la IP** (1,04), los **minutos desde la última transacción** (0,97) y el **dispositivo nuevo** (0,89), seguidas por los intentos fallidos de PIN y el indicador de operación nocturna. La hora aparece primero porque el modelo la usa de forma continua para reconocer la madrugada, que es cuando se concentra el fraude. El gráfico de puntos muestra que la dirección del efecto tiene sentido de negocio: valores altos de riesgo de IP y dispositivo nuevo empujan hacia fraude, mientras que pocos minutos desde la última transacción también lo hacen (por eso los puntos azules, de valor bajo, quedan a la derecha). Es decir, el modelo aprendió señales de **cómo** se hace la transacción y no de **quién** la hace: la edad, el segmento y el tipo de tarjeta aparecen al final del ranking.

9.2 Explicación de una alerta concreta

La transacción TX0179460 es una compra de 159,51 USD en un comercio de electrónica, hecha por App móvil el 30 de septiembre a las 4:09, en Guayaquil, que es la ciudad habitual del cliente. El modelo le asignó una probabilidad de fraude de prácticamente 1. La explicación muestra que lo que más pesó fueron los intentos fallidos de PIN, seguidos por el riesgo de la IP, el dispositivo nuevo, la hora de madrugada y el poco tiempo transcurrido desde la operación anterior. Es un buen ejemplo de que el modelo no se fija solo en el monto: la compra no es grande y ocurre en la ciudad del cliente, pero todo

lo demás delata un acceso no autorizado. Un analista puede leer esto en segundos y decidir si llama al cliente; un cliente que reclama puede recibir una explicación comprensible, que es justamente lo que exige la normativa.

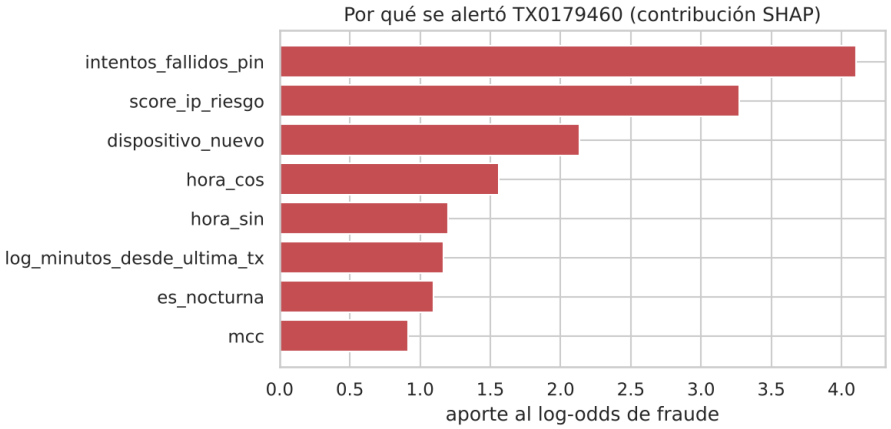


Figura 12. *Contribuciones SHAP de la alerta TX0179460.*

El mismo análisis aplicado al autoencoder mostró que este modelo se apoya sobre todo en el riesgo de la IP y en los intentos fallidos de PIN, sus dos primeras variables, y apenas usa el tipo de comercio, el dispositivo nuevo o el indicador nocturno, que sí son importantes para el boosting. Esto confirma que ambos modelos "miran" cosas distintas, lo que justifica combinarlos.

9.3 Implicaciones éticas

Aunque el boosting y el autoencoder pueden explicarse, siguen siendo modelos de caja negra: nadie puede leer sus 300 árboles o sus pesos y entender la regla completa. En decisiones financieras esto tiene consecuencias reales. Un falso positivo puede dejar a una persona sin poder pagar una medicina o un pasaje; si esos errores se concentran en ciertos grupos, el sistema discrimina aunque nadie lo haya programado así. En nuestro caso se tomaron tres precauciones: no usar la ciudad, los identificadores ni los nombres generados con Faker como variables, verificar que las variables demográficas tengan poco peso y mantener a una persona en la decisión final de los casos que no son evidentes. Aun así, con datos reales habría que medir las tasas de falsos positivos por provincia, edad y segmento, porque los datos históricos de un banco pueden subrepresentar a clientes rurales o de zonas con menos bancarización, y su comportamiento "distinto" podría confundirse con riesgo.

También hay que reconocer las limitaciones del trabajo. Los datos son sintéticos y los patrones de fraude fueron diseñados por nosotros, así que los modelos encontraron justamente lo que se puso; con datos reales las métricas serían más bajas y los fraudes más variados. Los tiempos se midieron en una CPU de dos núcleos sin Kafka ni Flink reales, y el costo de red es una estimación. Por eso los resultados deben leerse como una prueba de concepto del método, no como una promesa de desempeño en producción.

10. Conclusiones

El primer aprendizaje del proyecto es que, en detección de fraude, la forma de evaluar importa tanto como el modelo. Con una transacción fraudulenta por cada 134 legítimas, la exactitud y el F1 ponderado superan el 99 % para cualquier modelo y no dicen nada útil. Fue el AUC-PR el que permitió ver diferencias reales: entre 0,61 y 0,65 para los métodos no supervisados y 0,90 para el modelo supervisado. La partición temporal y la calibración del umbral en un periodo distinto al de la prueba fueron igualmente importantes para no engañarnos con resultados demasiado optimistas.

El segundo aprendizaje es práctico. Cuando existen etiquetas confiables, un modelo de gradient boosting es difícil de superar: detectó el 78 % del fraude con una precisión del 86 % y respondió en 134 ms por lote. Los métodos no supervisados no compiten en esas condiciones, pero cumplen un papel distinto e importante, porque son los únicos capaces de alertar sobre una modalidad de fraude que todavía no aparece en el historial. De ahí la propuesta de un sistema híbrido, en el que el autoencoder (el no supervisado más preciso y más rápido) actúa como red de seguridad; en la prueba, esta combinación logró el mejor F1 (0,825) y redujo las falsas alarmas de 33 a 22. LOF, aunque obtuvo un buen desempeño, quedó descartado por no cumplir el requisito de 500 ms.

(a) Relación con los requerimientos de la Superintendencia de Bancos. La Superintendencia exige a las entidades gestionar el riesgo operativo y la seguridad de la información en sus canales electrónicos, según la Norma de Control para la Gestión del Riesgo Operativo de su Codificación de normas, y ha pedido a los bancos documentar el cumplimiento de esa sección antes de habilitar nuevos servicios electrónicos (Superintendencia de Bancos del Ecuador, 2022). A esto se suma la nueva Ley Orgánica de Prevención, Detección y Combate del Delito de Lavado de Activos de 2024, que reemplazó a la ley de 2016 citada en la guía y mantiene la obligación de monitorear operaciones inusuales y reportarlas. El sistema propuesto responde a esas exigencias en tres puntos concretos: evalúa el 100 % de las transacciones antes de su autorización y no solo una muestra; deja un registro inmutable de cada score, umbral y explicación, que puede entregarse a un auditor o a la Unidad de Análisis Financiero y Económico; y documenta de forma reproducible cómo se construyó y validó el modelo, algo que un supervisor puede revisar. La calibración del umbral en función de la capacidad de los analistas también es una decisión de gestión de riesgo operativo, porque un sistema que genera más alertas de las que el banco puede revisar deja de proteger en la práctica.

(b) Relación con la Ley Orgánica de Protección de Datos Personales. Los datos transaccionales son datos personales, ya que revelan dónde está una persona, qué compra y cuánto gasta. La LOPDP (2021) obliga a tratarlos con una base legal, con minimización y con medidas de seguridad, y su artículo 20 reconoce el derecho a no ser objeto de decisiones basadas única o parcialmente en valoraciones automatizadas que afecten al titular. Además, en 2026 la Superintendencia de Protección de Datos Personales emitió una norma específica para el uso de inteligencia artificial que exige informar sobre el tratamiento automatizado, evaluar riesgos e impacto y auditar los sistemas según su nivel de riesgo. El diseño propuesto se ajusta a este marco: usa solo las variables necesarias para detectar fraude y excluye identificadores y ubicación como texto; las decisiones de bloqueo definitivo pasan por una persona en los casos dudosos; cada alerta lleva una explicación SHAP que permite responder al cliente en lenguaje claro; y el uso de datos sintéticos para el desarrollo demuestra que es posible diseñar y

probar el sistema sin exponer información real de clientes. Antes de un despliegue real, el banco debería realizar una evaluación de impacto e incluir este tratamiento en su registro de actividades.

(c) Potencial de escalabilidad en el sector financiero ecuatoriano. Los números juegan a favor. Un solo proceso en una CPU modesta puntuó más de 60.000 transacciones por segundo, mientras que la carga del banco es de 41 por segundo en promedio y de unas 200 en los picos estimados. Esto significa que la inferencia no es el cuello de botella; lo son la ingesta, el estado por cliente y la operación del sistema. Kafka y Flink escalan de forma horizontal añadiendo particiones y nodos, y son herramientas de código abierto, lo que reduce la barrera de entrada para cooperativas y bancos medianos, que en Ecuador tienen presupuestos tecnológicos mucho menores que los de los grandes bancos. La arquitectura también podría replicarse sobre el Sistema de Pagos Interbancarios del Banco Central para detectar patrones que ningún banco ve por sí solo, como una misma cuenta receptora que recibe transferencias de víctimas de distintas entidades. Los principales retos no son técnicos sino organizacionales: contar con etiquetas de calidad (fraudes confirmados y reclamos resueltos), personal capacitado para operar el streaming y una gobernanza clara del modelo que defina quién lo aprueba, quién lo monitorea y cuándo se reentrena.

En resumen, el trabajo muestra que un sistema de detección de fraude en tiempo real es viable con herramientas abiertas y dentro de las restricciones del caso, siempre que se combinen modelos adecuados, métricas honestas, una arquitectura pensada para la latencia y una explicación que permita a las personas entender y cuestionar cada alerta. Como pasos siguientes se propone validar el enfoque con datos reales anonimizados, incorporar un LSTM-Autoencoder sobre secuencias más largas y medir de forma sistemática la equidad del modelo entre provincias y segmentos de clientes.

Referencias

1. Asamblea Nacional del Ecuador. (2021). *Ley Orgánica de Protección de Datos Personales*. Registro Oficial Quinto Suplemento N.° 459, 26 de mayo de 2021.

2. Asamblea Nacional del Ecuador. (2024). *Ley Orgánica de Prevención, Detección y Combate del Delito de Lavado de Activos y de la Financiación de Otros Delitos*. Registro Oficial Cuarto Suplemento N.° 610, 29 de julio de 2024.

3. Branco, P., Torgo, L., & Ribeiro, R. P. (2016). A survey of predictive modeling on imbalanced domains. *ACM Computing Surveys, 49*(2), Artículo 31. https://doi.org/10.1145/2907070

4. Breunig, M. M., Kriegel, H.-P., Ng, R. T., & Sander, J. (2000). LOF: Identifying density-based local outliers. En *Proceedings of the 2000 ACM SIGMOD International Conference on Management of Data* (pp. 93–104). https://doi.org/10.1145/342009.335388

5. Carbone, P., Katsifodimos, A., Ewen, S., Markl, V., Haridi, S., & Tzoumas, K. (2015). Apache Flink: Stream and batch processing in a single engine. *Bulletin of the IEEE Computer Society Technical Committee on Data Engineering, 38*(4), 28–38.

6. Chandola, V., Banerjee, A., & Kumar, V. (2009). Anomaly detection: A survey. *ACM Computing Surveys, 41*(3), Artículo 15. https://doi.org/10.1145/1541880.1541882

7. Chawla, N. V., Bowyer, K. W., Hall, L. O., & Kegelmeyer, W. P. (2002). SMOTE: Synthetic minority over-sampling technique. *Journal of Artificial Intelligence Research, 16*, 321–357. https://doi.org/10.1613/jair.953

8. Davis, J., & Goadrich, M. (2006). The relationship between Precision-Recall and ROC curves. En *Proceedings of the 23rd International Conference on Machine Learning* (pp. 233–240). https://doi.org/10.1145/1143844.1143874

9. Gama, J., Žliobaitė, I., Bifet, A., Pechenizkiy, M., & Bouchachia, A. (2014). A survey on concept drift adaptation. *ACM Computing Surveys, 46*(4), Artículo 44. https://doi.org/10.1145/2523813

10. Ke, G., Meng, Q., Finley, T., Wang, T., Chen, W., Ma, W., Ye, Q., & Liu, T.-Y. (2017). LightGBM: A highly efficient gradient boosting decision tree. *Advances in Neural Information Processing Systems, 30*, 3146–3154.

11. Liu, F. T., Ting, K. M., & Zhou, Z.-H. (2008). Isolation forest. En *2008 Eighth IEEE International Conference on Data Mining* (pp. 413–422). https://doi.org/10.1109/ICDM.2008.17

12. Lundberg, S. M., & Lee, S.-I. (2017). A unified approach to interpreting model predictions. *Advances in Neural Information Processing Systems, 30*, 4765–4774.

13. Mehrabi, N., Morstatter, F., Saxena, N., Lerman, K., & Galstyan, A. (2021). A survey on bias and fairness in machine learning. *ACM Computing Surveys, 54*(6), Artículo 115. https://doi.org/10.1145/3457607

14. Pang, G., Shen, C., Cao, L., & van den Hengel, A. (2021). Deep learning for anomaly detection: A review. *ACM Computing Surveys, 54*(2), Artículo 38. https://doi.org/10.1145/3439950

15. Pedregosa, F., Varoquaux, G., Gramfort, A., Michel, V., Thirion, B., Grisel, O., Blondel, M., Prettenhofer, P., Weiss, R., Dubourg, V., Vanderplas, J., Passos, A., Cournapeau, D., Brucher, M., Perrot, M., & Duchesnay, É. (2011). Scikit-learn: Machine learning in Python. *Journal of Machine Learning Research, 12*, 2825–2830.

16. Ruff, L., Kauffmann, J. R., Vandermeulen, R. A., Montavon, G., Samek, W., Kloft, M., Dietterich, T. G., & Müller, K.-R. (2021). A unifying review of deep and shallow anomaly detection. *Proceedings of the IEEE, 109*(5), 756–795. https://doi.org/10.1109/JPROC.2021.3052449

17. Saito, T., & Rehmsmeier, M. (2015). The precision-recall plot is more informative than the ROC plot when evaluating binary classifiers on imbalanced datasets. *PLOS ONE, 10*(3), Artículo e0118432. https://doi.org/10.1371/journal.pone.0118432

18. Schölkopf, B., Platt, J. C., Shawe-Taylor, J., Smola, A. J., & Williamson, R. C. (2001). Estimating the support of a high-dimensional distribution. *Neural Computation, 13*(7), 1443–1471. https://doi.org/10.1162/089976601750264965

19. Štrumbelj, E., & Kononenko, I. (2014). Explaining prediction models and individual predictions with feature contributions. *Knowledge and Information Systems, 41*(3), 647–665. https://doi.org/10.1007/s10115-013-0679-x

20. Superintendencia de Bancos del Ecuador. (2022). *Circular N.° SB-INRE-2022-0021-C: Revisión y análisis de la seguridad de la información para las actividades financieras en canales electrónicos*.

21. Superintendencia de Protección de Datos Personales. (2026). *Resolución N.° SPDP-SPD-2026-0009-R: Norma general para la protección de datos personales en el uso de inteligencia artificial*.

22. Universidad Regional Autónoma de los Andes. (2026). *Modelos de inteligencia artificial para la detección de anomalías y predicción en tiempo real* [Material de clase, semana 2].